

Cetak Biru Pembaruan Arsitektur & Mitigasi Lonjakan Traffic

Dokumen Teknis Komprehensif Sistem Manajemen Tamu - sapatamu.id

Project:	sapatamu.id (Pembaruan Skala Sistem)
Target:	Development Agent / Team Engineer Backend & Frontend
Komponen Inti:	Supabase PostgreSQL, Google Apps Script (GAS), Frontend Client
Status Dokumen:	RELEASE PATCH v2.0 (FINAL)

Dokumen ini menyatukan strategi penanganan masalah **Connection Timeout** melalui restrukturisasi database, pembatasan muatan frontend, penerapan arsitektur **Database Replication & Failover**, serta penambahan sistem pertahanan keamanan **Rate Limiting (Token Bucket)**. Seluruh instruksi dalam cetak biru ini wajib diimplementasikan oleh tim pengembang guna mengamankan sistem saat fase beban puncak (peak hours).

1. Arsitektur Pertahanan: Rate Limiting (Token Bucket Algorithm)

Berdasarkan analisis ancaman lonjakan trafik mendadak (**burst traffic**), platform wajib mengimplementasikan gerbang pengaman di lapisan terdepan API sebelum **request** menyentuh Supabase. Tanpa adanya pembatasan (**Without Rate Limiting**), **burst traffic** akan langsung menghantam API, menyebabkan penumpukan pemrosesan (**backlog**), dan mengembalikan error `503 Service Unavailable`.

Solusi (With Rate Limiting): Pasang pembatas berbasis algoritma *Token Bucket*. Jika klien mengirim permintaan melebihi kuota per detik, sistem akan langsung menolak secara kilat dengan kode respons `429 Too Many Requests` tanpa membebani kinerja database.

Langkah Implementasi Rate Limiting pada Supabase Edge Functions:

```
import { Upstash } from "https://esm.sh/@upstash/ratelimit";
import { Redis } from "https://esm.sh/@upstash/redis";

const ratelimit = new Upstash.Ratelimit({
  redis: Redis.fromEnv(),
  limiter: Upstash.Ratelimit.tokenBucket(10, "1 s", 10),
  analytics: true,
});

export default async (req: Request) => {
  const ip = req.headers.get("x-forwarded-for") || "127.0.0.1";
  const { success, limit, reset, remaining } = await ratelimit.limit(ip);

  if (!success) {
    return new Response(JSON.stringify({ error: "Terlalu banyak permintaan scan. Coba sesaat lagi." }), {
      status: 429,
      headers: { "Content-Type": "application/json" },
    });
  }
};
```

2. Rekayasa Alur Sinkronisasi Supabase & GAS Google Sheets

Arsitektur pencatatan ganda saat ini menggunakan Supabase sebagai pangkalan utama dan Google Apps Script (GAS) Spreadsheet sebagai cadangan (*backup*). Untuk menghilangkan *timeout*, alur koordinasi kedua platform ini harus diubah.

PENTING: Larang keras frontend melakukan HTTP Request langsung ke GAS secara bersamaan dengan Supabase, atau memicu GAS langsung di dalam transaksi API utama. Latensi Google Sheets API sangat tinggi dan memicu pemutusan hubungan (timeout).

Alur Sinkronisasi Asinkron (Rekomendasi):

1. **Frontend Client** mengirim data check-in minimal (contoh: `{ qr_code: "T123", timestamp: 171820000 }`) ke Supabase API.
2. **Supabase PostgreSQL** memproses kueri, lalu secara instan (hitungan milidetik) mengembalikan status `200 OK` ke Frontend.
3. **Supabase Database Webhooks** menangkap perubahan data baru di tabel kehadiran secara otomatis, lalu mengirimkan payload data tersebut ke **Google Apps Script** di latar belakang (*background worker*).
4. Jika GAS mengalami kendala antrean, database utama Supabase dan kenyamanan user frontend sama sekali tidak akan terganggu.

3. Strategi Replikasi Database & Failover Kontingensi

Untuk mengantisipasi *Single Point of Failure* di mana database utama mati total, sistem harus dikonfigurasi mengikuti cetak biru sirkuit replikasi data:

- **Kondisi Normal (Synced):** Operasi Tulis (*Writes*) diarahkan penuh ke database **Primary Supabase**. Sementara, operasi Baca (*Reads*) seperti dashboard panitia atau pencarian manifes tamu diarahkan ke instans **Read Replica** guna membagi kapasitas beban kerja CPU.
- **Kondisi Darurat (Failover Active):** Jika server *Primary Supabase* mengalami *crash* atau tidak merespons akibat kendala infrastruktur cloud, Frontend harus memiliki sakelar logika otomatis (*fallback trigger*) untuk mengalihkan rute penulisan data langsung menuju gerbang **API Google Apps Script (Google Sheets)** secara sementara hingga server utama pulih.

4. Pembenahan Dasar Struktur Database & Frontend UI

A. Pengindeksan Database Keras

Jalankan perintah ini di Supabase SQL Editor untuk memastikan tidak ada pemindaian penuh tabel:

```
CREATE INDEX IF NOT EXISTS idx_sapatamu_qrcode ON tabel_tamu(qr_code);  
CREATE INDEX IF NOT EXISTS idx_sapatamu_status ON tabel_tamu(status_hadir);
```

B. Penggunaan Kueri Atomik Tunggal (Upsert)

Hilangkan penggunaan kueri bertingkat `SELECT` lalu `UPDATE`. Ganti dengan kueri satu aksi:

```
INSERT INTO tabel_kehadiran (tamu_id, waktu_hadir, status_hadir)  
VALUES ('ID_TAMU', NOW(), 'Hadir')  
ON CONFLICT (tamu_id)  
DO UPDATE SET waktu_hadir = NOW(), status_hadir = 'Hadir';
```

C. Mekanisme Antrean Lokal & Debounce Frontend

Fitur Frontend	Spesifikasi Teknis Wajib
Debounce Kamera/Scanner	Kunci fungsi penangkapan gambar QR selama 1.500 ms setelah kode terdeteksi pertama kali untuk menghindari *spam request*.
Antrean Lokal (IndexedDB)	Jika koneksi ke server melambat, masukkan data ke antrean lokal browser terlebih dahulu. Jangan biarkan layar *loading* membeku.

5. Daftar Periksa (Checklist) Evaluasi untuk Agent

Pengembang wajib mencentang seluruh komponen di bawah ini sebelum menyatakan patch selesai:

- ☐ Middleware Rate Limiter aktif di gerbang API (Merespons status 429 saat trafik tidak wajar).
- ☐ Logika pemisahan Read/Write kueri diatur dengan benar.
- ☐ Webhook asinkron dipasang dari Supabase ke GAS (bukan pemanggilan langsung dari client frontend).
- ☐ Indeks tabel PostgreSQL telah diterapkan dan divalidasi kinerjanya.